

Mobile Learning Application Development



Dart Programming Language Lecture 5

Aditya Rajbongshi
Assistant Professor

Dept. of Educational Technology and Engineering,
University of Frontier Technology, Bangladesh



Dart Object-Oriented Concepts

- Dart is an **object-oriented programming language**, and it supports all the concepts of object-oriented programming such as classes, object, inheritance, and abstract classes.
- The Object-oriented programming approach is used to **implement the concept like polymorphism, data-hiding, etc.**
- The main goal of oops is to **reduce programming complexity** and do **several tasks simultaneously**.
- The oops concepts are: **Class, Object, Inheritance, Polymorphism, Interfaces, Abstract class**



Dart Class

Dart classes are the blueprint of the object, or it can be called object constructors.

A class can contain fields, functions, constructors, etc.

It is a wrapper that binds/encapsulates the data and functions together; that can be accessed by creating an object.

Syntax:

```
class ClassName {  
    <fields>  
    <getters/setters>  
    <constructor>  
    <functions>  
}
```

```
class MyClass {  
    var stdId=111000;  
  
}
```



Creating Objects and Accessing Variable and Function

After creating the class, we can create an instance or object of that class which we want to access its **fields and functions**. The **new keyword** is used to declare class followed by the class name.

Syntax:

```
var object_name = new class_name(<constructor_arguments>);
```

```
class MyClass {  
    var stdId=111000;  
    var stdName="Ratul";  
    showInfo() {  
        print("Student Name is : ${stdName}");  
        print("Student Roll Number is : ${stdId}");  
    }  
}  
  
void main() {  
    MyClass ob1= new MyClass();  
    print(ob1.stdId);  
    ob1.showInfo();  
}
```



Dart Constructor

A constructor is a different type of function which is created with same name as its class name. The constructor is used to initialize an object when it is created.

- Constructor **has no return type**
- Constructor can have parameter
- Constructor **execute automatically**

Syntax:

```
class ClassName {  
    ClassName() {  
    }  
}
```

```
void main() {  
    Student std = new Student("Jones",26  
);  
}
```

```
class Student {  
    Student(String str, int age) {  
        print("The name is: ${str}");  
        print("The age is: ${age}");  
    }  
}
```



Default Constructor

A Constructor which has **no parameter** is called **default constructor** or **no-arg constructor**. It is **automatically** created (with no argument) by Dart compiler if we don't declare in the class. The Dart compiler ignores the default constructor if we create a constructor with argument or no argument. The syntax is given below.

Syntax:

```
class ClassName {  
    ClassName() {  
        // constructor body  
    }  
}
```

```
void main() {  
    Student std = new Student();  
}  
  
class Student {  
    Student() {  
        print("The example of the default constructor");  
    }  
}
```



Parameterized Constructor

We can also pass the parameters to a constructor that type of constructor is called parameterized constructor. It is used to initialize instance variables. Sometimes, we need a constructor which accepts single or multiple parameters. The parameterized constructors are mainly used to initialize instance variable with own values.

Syntax:

```
class ClassName {  
    ClassName(parameter_list)  
}
```

```
void main() {  
    Student std = new Student("Jones",26  
);  
}  
class Student{  
  
    Student(String str, int age){  
        print("The name is: ${str}");  
        print("The age is: ${age}");  
    }  
}
```



Named Constructor

The named constructors are used to declare the **multiple constructors** in single class.

Syntax:

className.constructor_name(param_list)

```
void main() {  
    Student std1 = new Student();  
    Student std2 = new Student.namedConst("Computer Science");  
}  
  
class Student{  
  
    Student(){  
        print("The example of the named constructor");  
    }  
  
    Student.namedConst(String branch){  
        print("The branch is: ${branch}");  
    }  
}
```



Dart : this Keyword

✓ The this keyword is used to refer the **current class object**.

✓ It indicates the current instance of the class, methods, or constructor.

✓ It can be also used to **call** the current **class methods or constructors**.

✓ It **eliminates** the uncertainty between **class attributes and the parameter names are the same**.

```
class Mobile {  
    String modelname;  
    int man_year;  
    Mobile(modelname, man_year){  
        this.modelname = modelname;  
        this.man_year = 2020;  
        print("Mobile's model name is: ${modelname}, and the manufacture year is: ${man_year}");  
    }  
}  
  
void main(){  
    Mobile mob = new Mobile("iPhone 11",2020);  
}
```



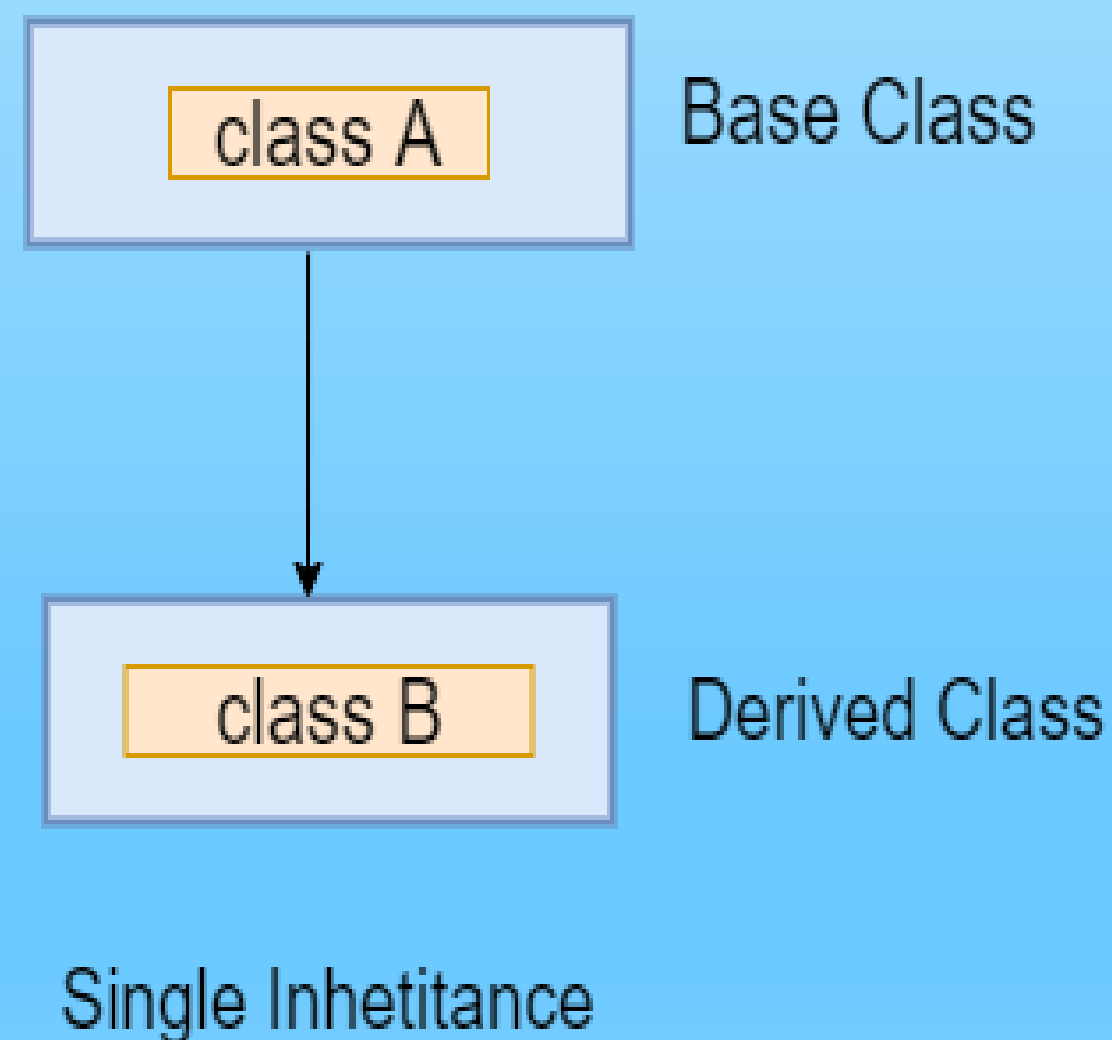
Inheritance

- Dart inheritance is defined as the process of deriving the **properties** and **characteristics** of **another class**.
- It provides the ability to create **a new class** from an **existing class**.
- It is the most **essential concept** of the oops(Object-Oriented programming approach).
- We can **reuse** the all the **behavior** and **characteristics** of the **previous class** in the new class.
- **Parent Class:**
A class which is inherited by the other class is called superclass or parent class. It is also known as a base class.
- **Child Class:**
A class which inherits properties from other class is called the child class. It is also known as the derived class or subclass.



Single Inheritance

In the single inheritance, a class is inherited by a single class or subclass is inherited by one parent class.

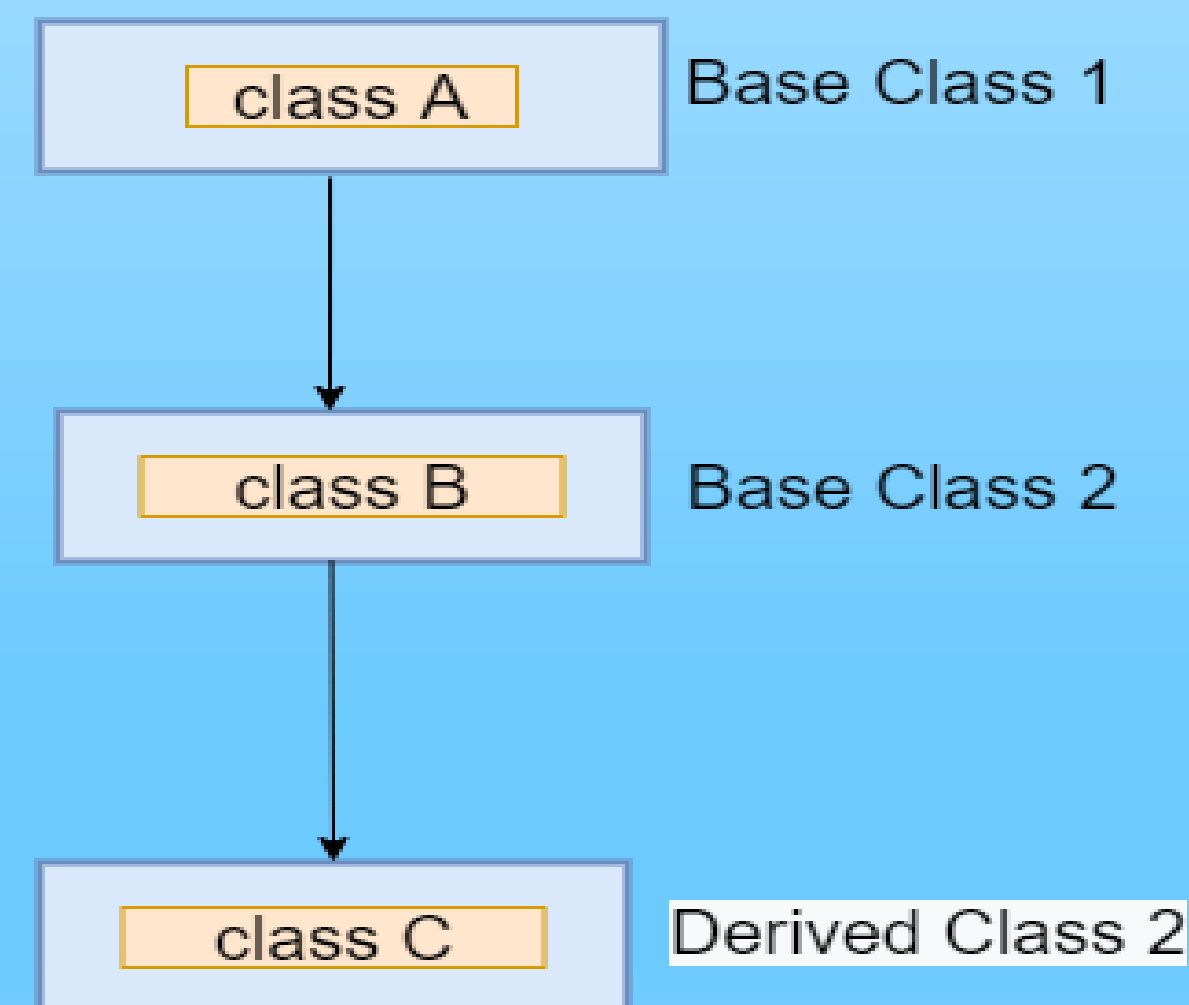


```
class Bird{
    void fly()
    {
        print("The bird can fly");
    }
}
class Parrot extends Bird{
    void speak(){
        print("The parrot can speak");
    }
}
void main() {
    Parrot p=new Parrot();
    p.speak();
    p.fly();
}
```



Multilevel Inheritance

In the multiple inheritance, a subclass is inherited by another subclass or creates the chaining of inheritance.



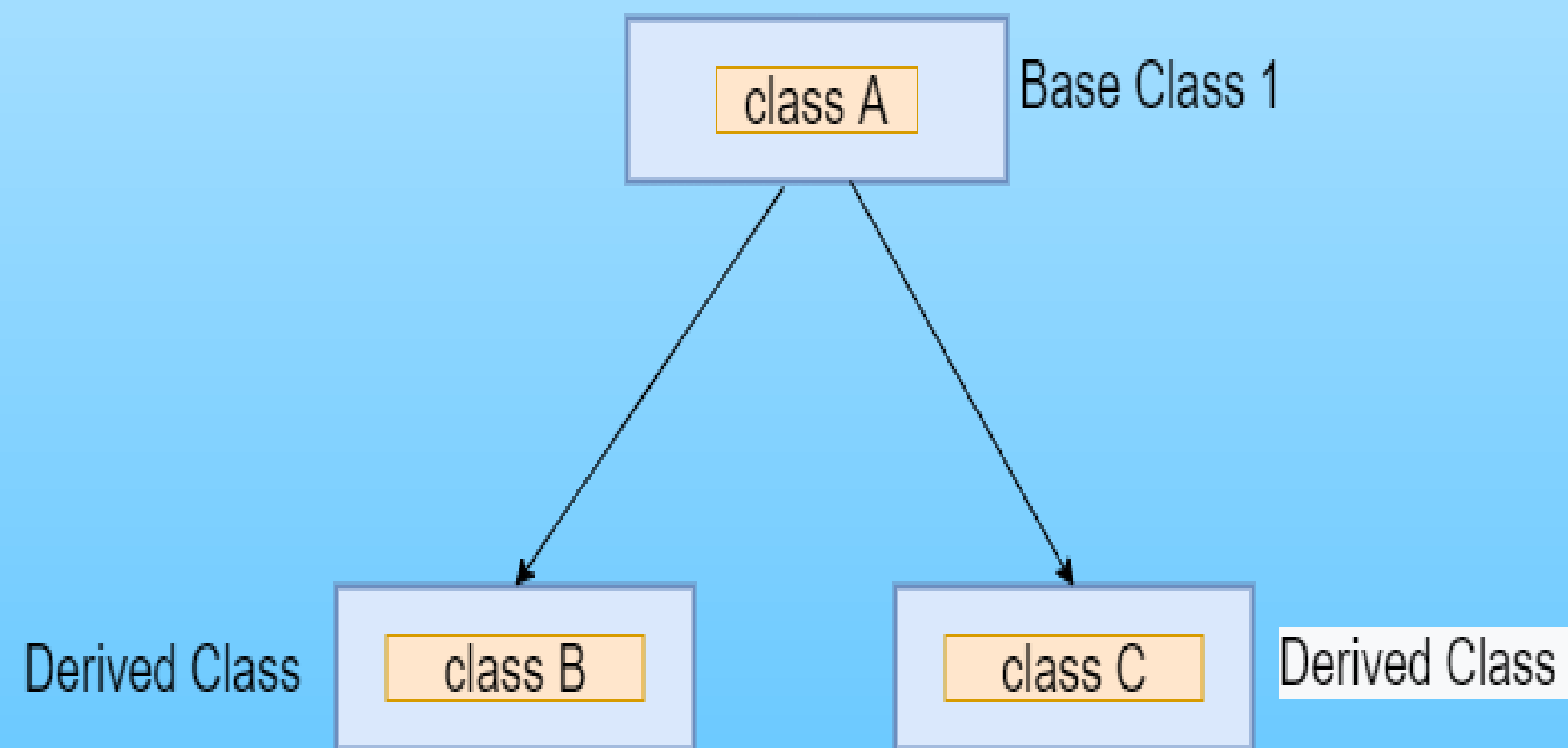
Multilevel Inheritance

```
class Bird{  
  
}  
  
class Parrot extends Bird{  
  
}  
  
class Eagle extends Parrot {  
  
}  
  
void main() {  
  
}
```



Hierarchical Inheritance

In the hierarchical inheritance, two or more classes inherit a single class.



```
class Person {  
}
```

```
class Peter extends Person {  
}
```

```
class James extends Person {  
  
}
```

```
void main() {  
  
}
```



Hierarchical Inheritance

- The super keyword is used to denote the instant parent class object of the current child class.
- It is used to invoke **superclass methods**, **superclass constructor** in its child class.
- Syntax:

Super.varName

super.methodName;

```
class Parent
{
    Parent()
    {
        print("This is the super class constructor");
    }
}
class Child extends Parent
{
    Child():super()
    {
        print("This is the sub class constructor");
    }
}
void main() {
    Child c = new Child();
}
```



Method Overriding

When we declare the **same method** in the **subclass**, which is previously defined in the superclass is known as the **method overriding**.

The subclass can **define the same method** by providing its own implementation, which is already exists in the superclass.

The method in the superclass is called method overridden, and method in the subclass is called method overriding.

```
class Human{  
    void run()  
    {  
        print("Human is running");  
    }  
}  
class Man extends Human{  
    void run(){  
        print("Boy is running");  
    }  
}  
void main(){  
    Man m = new Man();  
    m.run();  
}
```



Method Overriding using super keyword

- . It can **invoke the parent class method** without creating its object.
- . It can be done by using the **super keyword** in the subclass.
- . The parent class data member can be accessed in the subclass by using the **super** keyword.

```
class Human{
    void run()
    {
        print("Human is running");
    }
}

class Man extends Human{
    void run(){
        super.run();
        print("Boy is running");
    }
}

void main(){
    Man m = new Man();
    //This will call the child class version of eat()
    m.run();
}
```



Dart Getters and Setters

- . Getters and setters are the **special class method** that is used to read and write access to an object's properties.
- . The getter method is used to reads the value of the variable or retrieve the value and setter method is used to set or initialize respective class fields.
- . By default, all classes are **associated with getter and setter method**.

```
class Student {  
    String stdName;  
    String branch;  
    int stdAge;  
    String get std_name  
    {  
        return stdName;  
    }  
    void set std_name(String name)  
    {  
        this.stdName = name;  
    }  
}  
void main(){  
    Student std = new Student();  
    std.std_name = 'John';  
    print("Student name is: ${std_name}");  
}
```



Abstract Classes

- Abstract classes are the classes in Dart that has **one or more abstract method**.
- Abstraction is a part of the **data encapsulation** where the actual internal working of the function hides from the users.
- Declare the abstract class by using the **abstract** keyword.
- There is a **possibility** that an abstract class **may or may not have abstract methods**.
- **Abstract methods** are those methods, which are declared **without implementation**.
- The **concrete methods** or normal methods are declared **with implementation**.
- An abstract class can contain both types of methods, but **a normal class is not allowed to have abstract methods**.



Declaring Abstract Class

An abstract keyword followed by a class name is used to declare the abstract class.

Syntax:

```
abstract class ClassName {  
    // Body of abstract class  
}
```

```
abstract class Person {  
    void displayInfo();  
}  
class Boy extends Person  
{  
    void displayInfo() {  
        print("My name is Johnathon");  
    }  
}  
void main() {  
    Boy b = new Boy();  
    b.displayInfo();  
}
```



Dart Interfaces

- An interface defines the syntax that any entity must adhere to.
- Dart does not have any separate syntax to define interfaces.
- An Interface defines the same as the class where any set of methods can be accessed by an object.
- The Class declaration can interface itself.
- The keyword `implements` is needed to be writing, followed by class name to be able to use the interface.

Syntax:

class ClassName **implements InterfaceName**

```
class Employee
{
    void display() {
        print("I am working as an engineer");
    }
}

class Engineer implements Employee
{
    void display() {
        print("I am an engineer in this company");
    }
}

void main()
{
    Engineer eng = new Engineer();
    eng.display();
}
```